Lenguajes de Programación

Primera Prueba Presencial

INTRODUCCIÓN + PROGRAMACIÓN FUNCIONAL

Fecha: 27 de octubre de 2014

Valor de la prueba: 33%

Advertencia Importante: Observa bien la sintaxis en los ejemplos. No se considerarán soluciones que utilicen funciones auxiliares y/o no mantengan la sintaxis de la llamada y de los resultados.

EJERCICIO 1 (2,5 puntos)

Sin definir otras funciones auxiliares, definir una función en lisp llamada **aplicar** que acepte un número indeterminado de valores seguido de una función y devuelva una lista con el resultado de aplicar la función sobre cada uno de los valores.

```
(aplicar 1 2 3 (lambda(x)(+ 1 x)))
> (2 3 4)
(aplicar #'1+)
> NIL
(aplicar 9 6 8 #'1+)
> (10 7 9)
```

EJERCICIO 2 (2,5 puntos)

Sin definir otras funciones auxiliares, escribe una función en LISP llamada **sumamult** que acepte como único parámetro una lista de valores (positivos o negativos), y devuelva una lista de dos elementos con la suma y la multiplicación de los valores (en este orden). La función deberá devolver NIL si la lista proporcionada como parámetro estuviera vacía.

```
> (sumamult '())
NIL
> (sumamult '(8))
(8 8)
> (sumamult '(2 8))
(10 16)
> (sumamult '(1 3 2 4))
(10 24)
> (sumamult '(1 3 -2 4))
(6 -24)
```

EJERCICIO 3 (2,5 puntos)

Sin definir otras funciones auxiliares, escribe una función en lisp llamada sumafunciones que acepte un número indeterminado de funciones unarias seguidas de un valor numérico y devuelva la suma resultante de la aplicación de cada una de las funciones sobre el valor.

```
> (sumafunciones 5) ; Solo el valor. Ninguna función
0
> (sumafunciones #'1+ (lambda(x)(- 0 x))  5)
1
> (sumafunciones #'1+ #'1+ 2)
6
> (sumafunciones #'1+ (lambda(x)(+ 1 x)) 2)
6
> (sumafunciones #'1+ (lambda(x)(* 2 x)) 2)
7
> (sumafunciones #'1+ #'1+ #'1+ #'1+ 2)
12
```

EJERCICIO 4 (2,5 puntos)

Sin definir otras funciones auxiliares, define una función en LISP llamada **tamanyo** que acepte una lista como parámetro y devuelva una nueva lista con tres elementos. El primero será el número de elementos que contiene. El segundo el número de elementos mayor que cero y el tercero el número de elementos menores que cero.

```
> (tamanyo '())
(0 0 0)
> (tamanyo '(1 2 3 4 5))
(5 5 0)
> (tamanyo '(1 2 3 4 -1 -1 5))
(7 5 2)
> (tamanyo '(-1))
(1 0 1)
> (tamanyo '(1 2 0 3 4 0 -1 -1 0 0 5))
(11 5 2)
```

EJERCICIO 4 (3 puntos)

Sin definir otras funciones auxiliares, define una función en LISP llamada **ordenar** que acepte un número indefinido de valores numéricos y devuelva una lista con tres elementos. El primer elemento debe ser una lista que contenga los elementos originales en el mismo orden en que fueron proporcionados como argumentos. El segundo debe ser una lista con los elementos ordenados de menor a mayor y el último una lista con los elementos ordenados de mayor a menor.

```
> (ordenar NIL)
(NIL NIL NIL)
> (ordenar 3)
((3) (3) (3))
> (ordenar 2 5 3)
((2 5 3) (2 3 5) (5 3 2))
```